

Assistente de Compras Inteligente

Documentação Técnica Completa

Disciplina: Programação II — 2025

Projeto: Assistente de Compras Inteligente

Data: 08/06/2026

1. Introdução

O Assistente de Compras Inteligente é um sistema desenvolvido na disciplina de Programação II que interpreta buscas escritas em linguagem natural e retorna recomendações de produtos de forma organizada e eficiente.

A motivação do projeto parte de um problema cotidiano: o usuário muitas vezes não sabe o nome exato do produto que quer. Ele pesquisa "celular barato com boa câmera" ou "notebook da Samsung para trabalho". O sistema interpreta essa intenção e apresenta resultados relevantes.

Objetivos

- Interpretar buscas em linguagem natural (NLP)
- Organizar produtos com estruturas de dados eficientes
- Gerar recomendações inteligentes baseadas em relações e histórico
- Reconhecer categorias, preços, atributos e marcas de produtos
- Aplicar na prática as estruturas de dados estudadas na disciplina

2. Arquitetura e Fluxo de Processamento

O sistema processa cada busca em uma sequência de etapas bem definidas, onde cada estrutura de dados tem uma responsabilidade específica.

Fluxo completo de uma busca

1. Usuário digita a busca na interface web
2. Fila Simples (FIFO) recebe e ordena as requisições
3. NLP Parser interpreta a frase e extrai: categoria, preço, marca, atributos
4. Fila de Prioridade ordena a busca por relevância (score)
5. Árvore de Categorias localiza os produtos correspondentes
6. Hash Table permite acesso direto a qualquer produto por ID
7. Grafo de Produtos gera recomendações ("quem comprou X também comprou Y")
8. Pilha armazena o histórico e sugere produtos baseados em buscas anteriores
9. Resultados são retornados via API JSON para o frontend

3. Estruturas de Dados

3.1 Fila Simples — FIFO (First In, First Out)

Arquivo: estruturas/fila_simples.py

Primeira estrutura a receber as entradas do usuário. Garante que todas as requisições sejam processadas na ordem exata de chegada, sem perda de dados em cenários de múltiplas requisições simultâneas.

Implementada com `collections.deque` para operação $O(1)$ nas duas extremidades.

- `enqueue(req)`: adiciona requisição ao final da fila — $O(1)$
- `dequeue()`: remove e retorna a requisição mais antiga — $O(1)$
- `esta_vazia()`, `tamanho()`: consultas de estado

3.2 Fila de Prioridade (Max-Heap)

Arquivo: `estruturas/fila_prioridade.py`

Após o NLP processar a busca, ela é inserida na fila de prioridade. Buscas mais específicas (com mais filtros) recebem maior prioridade e são processadas primeiro, melhorando a eficiência do sistema.

Implementada com `heapq` (min-heap com negação de prioridade para simular max-heap).

Critérios de pontuação

- +3 pontos — categoria identificada (celular, notebook, tv...)
- +2 pontos — faixa de preço identificada (baixo, médio, alto)
- +2 pontos — marca identificada (Samsung, Apple, Sony...)
- +1 ponto — por cada atributo identificado (câmera, bateria...)

3.3 Pilha (Stack — LIFO)

Arquivo: estruturas/pilha.py

Armazena o histórico de buscas do usuário. Permite navegar para buscas anteriores (funcionalidade "Voltar") e gera recomendações baseadas nos padrões de busca recentes. Capacidade configurável (padrão: 50 entradas).

Implementada com `collections.deque(maxlen=capacidade)` — remoção automática $O(1)$.

- `push(item)`: empilha a busca atual
- `pop()`: remove e retorna o item do topo
- `voltar()`: descarta a busca atual e retorna a anterior
- `historico(n)`: retorna as últimas n buscas
- `categorias_recentes(n)`: retorna as categorias das últimas n buscas

3.4 Hash Table (Tabela de Dispersão)

Arquivo: estruturas/hash_table.py

Armazena o catálogo completo de produtos com acesso direto por ID em $O(1)$ médio. Utiliza encadeamento (chaining) para resolver colisões. Capacidade padrão de 100 buckets com fator de carga baixo dado o tamanho do catálogo.

- `inserir(id, produto)`: armazena ou atualiza um produto — $O(1)$ médio
- `buscar(id)`: acesso direto por ID — $O(1)$ médio
- `buscar_por_atributo(campo, valor)`: filtro por qualquer campo — $O(n)$
- `listar_todos()`, `listar_chaves()`, `listar_itens()`: enumeração completa

3.5 Árvore de Categorias (N-ária)

Arquivo: estruturas/arvore.py

Organiza os produtos em hierarquia de categorias, permitindo navegação e busca eficiente por ramos. A estrutura atual é: eletrônico → celular / notebook / tv / fone / tablet / smartwatch / console / monitor.

- `inserir(caminho, produto)`: insere produto no nó folha do caminho
- `buscar_categoria(caminho, filtros)`: retorna todos os produtos do ramo, com filtros opcionais
- `categoria_existe(caminho)`: verifica se um caminho de categorias existe

3.6 Grafo de Produtos (Não-dirigido com Pesos)

Arquivo: estruturas/grafos.py

Representa relações de co-compra entre produtos ("quem comprou X também comprou Y"). Cada aresta tem um peso entre 0.0 e 1.0 que indica a força da relação. Implementado como dicionário de adjacência.

- `adicionar_relacao(a, b, peso)`: cria aresta bidirecional com peso
- `recomendar(id, top_n)`: retorna os top_n vizinhos de maior peso
- `recomendar_por_historico(ids)`: agrega vizinhos de múltiplos produtos do histórico
- `produto_existe(id)`, `listar_relacoes()`, `grau(id)`: consultas estruturais

4. NLP e Parser

Arquivo: `nucleo/nlp_parser.py`

O NLPParser é responsável por interpretar a intenção do usuário a partir de texto livre. Funciona com correspondência de padrões (sem modelos de ML externos) e normalização de texto para lidar com acentos, plurais e variações de gênero.

Normalização

Todo texto é normalizado antes da comparação: convertido para minúsculas e acentos removidos via `unicodedata.normalize()`. Isso garante que "câmera", "camera" e "CÂMERA" sejam tratados igualmente, e que "barata" corresponda a "barato".

Campos extraídos

Campo	Exemplos reconhecidos	Resultado
tipo	quero, comprar, busco, versus, o que é	compra / comparar / informacao
categoria	celular, celulares, smartphone, iphone	celular
categoria	notebook, notebooks, laptop, pc	notebook
categoria	tv, tvs, televisao, televisoes	tv
categoria	tablet, tablets, ipad	tablet
categoria	console, ps5, xbox, nintendo	console
preco	barato, barata, econômico, acessível	baixo
preco	intermediário, médio, moderado	medio
preco	premium, caro, cara, topo de linha	alto
marca	samsung, apple, xiaomi, sony, dell	samsung / apple / ...
atributos	câmera, bateria, tela, memória	lista de atributos

Marcas reconhecidas (24)

samsung, apple, motorola, xiaomi, sony, lg, dell, acer, hp, asus,
lenovo, positivo, jbl, beats, edifier, tcl, philips, nintendo, microsoft,
garmin, amazfit, aoc, multilaser, realme

5. Catálogo de Produtos

Arquivo: `dados/catalogo.py`

O catálogo é um módulo separado com 49 produtos em 8 categorias. Cada produto tem um ID único, nome, categoria, marca, faixa de preço e valor em reais. As relações do grafo também são definidas aqui (40+ arestas).

Estrutura de um produto

Campos de cada produto

id — identificador único (ex: 'cel003')

nome — nome comercial (ex: 'iPhone 15')

categoria— grupo de produto (ex: 'celular')

marca — fabricante (ex: 'apple')

preco — faixa: 'baixo', 'medio' ou 'alto'

valor — preço em reais (ex: 5999.00)

Categoria	Qtd.	Marcas presentes
celular	9	Motorola, Samsung, Apple, Xiaomi, Realme
notebook	8	Acer, Dell, Apple, Lenovo, HP, ASUS, Samsung, Positivo
tv	6	TCL, Samsung, LG, Philips, Sony, AOC
fone	7	JBL, Sony, Beats, Edifier, Samsung, Apple, Multilaser
tablet	5	Samsung, Apple, Lenovo
smartwatch	5	Xiaomi, Samsung, Apple, Amazfit, Garmin
console	4	Nintendo, Sony, Microsoft
monitor	5	AOC, LG, Dell, Samsung, Multilaser

6. Interface Web e Servidor

Servidor HTTP (servidor.py)

Servidor Python puro usando `http.server.HTTPServer`. Inicializa o `AssistenteDeCompras` uma vez e mantém o estado (histórico) entre requisições.

Rota	Método	Descrição
/	GET	Serve o frontend (index.html)
/buscar?q=	GET	Processa busca e retorna JSON com resultados e recomendações
/voltar	GET	Retorna resultados da busca anterior (usa a Pilha)

Frontend (index.html)

Interface web em HTML/CSS/JavaScript puro, sem frameworks externos. Design escuro com identidade visual roxa. Funcionalidades principais:

- Barra de busca com sugestões rápidas (chips de categorias)
- Resultados separados em duas seções: Resultados diretos e Recomendados
- Botão "Voltar" que usa o histórico da pilha para retornar à busca anterior
- Mensagem de não encontrado com chips de categorias sugeridas
- Ícones por categoria, badge de faixa de preço com cores distintas
- Suporte a Enter na barra de busca e animação de entrada dos cards

7. Estrutura de Arquivos

Organização do projeto

Assistente-de-Compras-Inteligente/

— assistente.py	— classe principal AssistenteDeCompras
— servidor.py	— servidor HTTP com rotas /buscar e /voltar
— index.html	— interface web (frontend)
— dados/	
— catalogo.py	— 49 produtos e 40+ relações do grafo
— estruturas/	
— fila_simples.py	— Fila FIFO com deque
— fila_prioridade.py	— Max-Heap com heapq
— pilha.py	— Stack com deque(maxlen)
— hash_table.py	— Tabela de dispersão com chaining
— arvore.py	— Árvore N-ária de categorias
— grafo.py	— Grafo não-dirigido com pesos
— nucleo/	
— nlp_parser.py	— Parser de linguagem natural
— testes/	

- |— test_fila_simples.py
- |— test_fila_prioridade.py
- |— test_hash_table.py
- |— test_pilha.py
- |— teste_grafo.py
- |— teste_arvore.py

8. Como Executar

Pré-requisito: Python 3.x instalado. Nenhuma biblioteca externa é necessária.

Passos para iniciar

1. Abra o terminal na pasta do projeto:
`cd "Projeto Programação 2/Assistente-de-Compras-Inteligente"`
2. Execute o servidor:
`python servidor.py`
3. O navegador abre automaticamente em `http://localhost:8000`
4. Para parar o servidor: Ctrl+C no terminal

Executar os testes

Rodar testes individuais

```
python testes/test_fila_simples.py
python testes/test_pilha.py
python testes/test_hash_table.py
python testes/teste_grafo.py
python testes/teste_arvore.py
```